Re-examining ASSERT: An Open-Model Replication and a Scenario-Level Re-analysis of Released Outputs

Rohail Ahmad
Independent Researcher
priv.rohail6796@gmail.com

Abstract

ASSERT [8] tests the robustness of language models on physical-safety questions from SafeText [6] with three automatic red-teaming methods: semantically aligned augmentation (SAA), targeted bootstrapping (TB) and adversarial knowledge injection (AKI). It compares each method against the original benchmark with a two-proportion z-test. We make two contributions. First, we replicate all three methods with an open 4-bit model, Qwen2.5-7B-Instruct-AWQ [10], on a free Colab T4 GPU, with pinned code, data and weights. Second, we re-analyse the outputs the ASSERT authors released for GPT-3.5, GPT-4, Alpaca and Vicuna. The released paraphrases do not record their source scenario, so we recover the link with an order-preserving aligner that is exact on unsafe items in simulation. From the released files we reproduce all 16 overall effects in the paper's Tables 2–4 to within 0.03 points, the precision of its printed values. We then compare each method with the baseline on the *same* scenarios and resample whole scenarios rather than single items. AKI's large effect survives in every setting, and in our replication it raises Qwen's error on unsafe scenarios from 4.6% to 24.6% (+20.0 points, 95% CI [16.1, 24.0]). Two published conclusions change. Vicuna's significant SAA effect on unsafe scenarios (-7.27 accuracy points, $p < .01$) shrinks to +2.7 error points (95% CI $[-0.8, 6.2]$) once Vicuna is compared on the 245 of 370 scenarios for which it produced paraphrases. GPT-3.5's non-significant SAA effect on unsafe scenarios ($p = 0.23$) becomes borderline significant ($p = 0.04$) under a paired analysis. We also find that the change in error reported for cross-model attacks appears to be computed against the source model's baseline rather than the target's. We release code, the run manifest and all aggregate results.

1 Introduction

Red-teaming methods for language models are usually judged by how much they raise a model's error rate over a benchmark. ASSERT [8] is a clear example. Starting from SafeText [6], a set of everyday scenarios paired with safe and unsafe advice, it generates new test items in three ways: paraphrasing (SAA), writing new related scenarios (TB), and injecting model-written "benefits" of the unsafe action as hints (AKI). It reports per-domain two-proportion z-tests against the original benchmark.

Generated test items are not independent draws. Five paraphrases of one scenario share its difficulty, and a generator that fails on some scenarios produces a test set that covers only part of the benchmark. Both properties affect the statistics. Correlated items make an unpaired test either too cautious or too confident. A partial test set compared against the full benchmark confounds the method's effect with a change in which scenarios are being tested. These issues are well known in statistics [2, 3] and have recently been raised for language-model evaluations [1, 9], but they are rarely checked in red-teaming papers.

This paper asks two questions. (1) Do ASSERT's findings hold for a small, open, quantized model run on free hardware? (2) Do the published conclusions change when the released outputs are analysed at the level of scenarios? Our contributions are:

- An open, reproducible replication of SAA, TB and AKI with Qwen2.5-7B-Instruct-AWQ, served with vLLM [5], with every deviation from the original protocol listed (Section 3).
- An aligner that links the authors' released paraphrases to their source scenarios, validated by simulation (Section 5). With it, we reproduce the paper's reported overall effects from the released files to within 0.03 points, the precision of its printed values.
- A paired, scenario-level re-analysis of the released outputs, which changes two published conclusions and identifies a baseline mismatch in the cross-model table (Section 6.2).
- Concrete recommendations for evaluating generated red-teaming data (Section 7).

2 Background

SafeText. SafeText contains 1,095 safe and 370 unsafe pieces of advice, each attached to a scenario ("If you're on a hike in the woods and you see a colorful mushroom, should you taste it?"). Scenarios fall into four domains, which ASSERT calls Outdoors, Medical, Household and Extra. Several pieces of safe advice can share one scenario context.

ASSERT. A model is asked each question in a four-shot Yes/No format with a short rationale. The correct label is "Yes" for safe advice and "No" for unsafe advice. **SAA** asks a model for five paraphrases of each question and evaluates the model on them. **TB** isolates the action of an unsafe scenario and asks for new contexts, then asks for new unsafe actions in those contexts. The authors filtered the results by human rating and released 3,564 scenarios. **AKI** asks a model for "benefits" of each unsafe action and injects them as hints, zero-shot or with adversarial four-shot examples. Self-attacks use benefits written by the evaluated model. Cross-model attacks use benefits written by GPT-3.5 or GPT-4 against Alpaca and Vicuna. Each method is compared against the original benchmark with a two-tailed two-proportion z-test at $\alpha = 0.05$, per domain and overall.

3 Replication setup

Model and serving. We use Qwen2.5-7B-Instruct with 4-bit AWQ quantization [7, 10], Hugging Face revision `b2503754`, served by vLLM on one Tesla T4 (16 GB) in Google Colab. Decoding is greedy (temperature 0, top-p 1, no penalties, at most 256 new tokens).

Prompts and scoring. Prompt text is taken from the authors' repository at commit `1cafa82`, used with their permission. The first word of the answer is the label, matched by `^\W*(yes|no)\b` without regard to case. An answer that does not start with Yes or No counts as an error.

Deviations. Table 1 lists every change. The main ones are a smaller TB expansion (3 new contexts $\times$ 3 new actions instead of 5 $\times$ 5), a stricter parser for TB output because Qwen often ignored the list delimiter, and a filter that drops junk or duplicate AKI benefits. TB scenarios generated by Qwen were *not* filtered by humans, unlike the authors' released TB set. We therefore report TB twice, on Qwen's own scenarios and on the authors' released scenarios.

Reproducibility. The full pipeline runs from one notebook in about an hour. A manifest records the code commit, the authors' commit, the model revision, package versions, the GPU and SHA-256 hashes of every output file. An integrity check (complete counts, no duplicates, no corrupt cache lines) passed on the reported run. We ran the pipeline twice: once during development and once from an empty cache after fixing the AKI filter. All effects agree to within 0.7 points. We report the second, clean run.

	Original protocol (kept)	This replication (changed)
Model	GPT-3.5, GPT-4, Alpaca, Vicuna	Qwen2.5-7B-Instruct-AWQ (4-bit)
Evaluation	4-shot Yes/No with rationale; z-test, $\alpha = 0.05$	First word is the label; unparsed counts as error
SAA	5 paraphrases per question	Keep a paraphrase only if it ends in "?" and does not start with what/how/why
TB	Contexts from an action, then actions per context; human rating filter	3×3 expansion; strict phrase parser; no human filter
AKI	Model-written benefits as hints; 0- and 4-shot	Junk and duplicate benefits dropped (1,181 items)
Prompt order	System message appended last	System message first
Statistics	Two-proportion z-test	Added: paired cluster bootstrap over scenarios

Table 1: What was kept from ASSERT and what was changed.

4 Statistical analysis

Scenario-level bootstrap. Each generated item comes from a source scenario. For SAA and AKI we use a *paired* cluster bootstrap [3]. We keep the source scenarios that have at least one generated item, resample them with replacement, and recompute both the baseline error (on the resampled originals) and the method error (on all their generated items). The difference of the two rates is the effect. TB creates new scenarios, so we resample baseline scenarios and TB contexts independently, which is an unpaired cluster bootstrap. We use 10,000 resamples, percentile 95% intervals, and the two-sided $p = 2\min(P(\hat{d} \leq 0), P(\hat{d} \geq 0))$, floored at 10^{-4}. Effects are reported in error points (method error minus baseline error). The paper reports SAA and TB as changes in accuracy, so its signs are flipped.

Two differences from the z-test. The paired analysis (i) compares like with like, because the baseline is computed only on scenarios that the method actually covers, and (ii) treats a scenario's variants as one unit. The first matters when generation fails unevenly across scenarios. The second usually makes the paired test *more* powerful, because baseline and method errors on the same scenario are positively correlated and the pairing removes that shared variance.

Multiple comparisons. Each analysis contains about 30 per-domain tests. We treat per-domain results as exploratory and base conclusions on the overall comparisons, applying Holm's correction [4] within each family of overall tests.

5 Linking released paraphrases to their sources

The authors released every model answer, but the paraphrase files contain only the paraphrase and its domain. For GPT-3.5 and GPT-4 the files hold exactly five paraphrases per scenario (5,475 = 1,095 × 5 safe, 1,850 = 370 × 5 unsafe), and stored order matches source order: every paraphrase's domain matches its source's, and word overlap with the assigned source is about 80 times that with a random source. Alpaca and Vicuna files are shorter (5,357 and 4,593 safe, 1,834 and 1,078 unsafe) because some paraphrases were dropped.

We align them with a Viterbi search that assigns each paraphrase to a source. The assignment must be non-decreasing in source order, use each source at most five times and respect domains, and it maximises the Jaccard overlap of content words between paraphrase and source. We validate it on the GPT-3.5 files, where the truth is known, after deleting items. Table 2 shows the result. With random item deletion, or with a third of all unsafe scenarios removed, which mimics Vicuna, unsafe items are recovered exactly and so is the number of covered scenarios.

Split	Deletion	Exact source	Same context
Unsafe	20% of items	1.000	1.000
Unsafe	45% of items	1.000	1.000
Unsafe	∼1/3 of scenarios + 15% of items (3 seeds)	1.000	1.000
Safe	20% of items	0.967	1.000
Safe	45% of items	0.935	1.000
Safe	∼10% of scenarios + 10% of items (3 seeds)	0.975–0.976	1.000

Table 2: Aligner validation on GPT-3.5 paraphrase files with simulated deletions.

Method	n	Base err. %	Method err. %	Δ [95% CI]	p (bootstrap)	p (z-test)
AKI 4-shot (unsafe)	1,181	4.63	24.64	+20.01 [16.11, 24.04]	<0.0001	<0.0001
AKI 0-shot (unsafe)	1,181	4.63	8.98	+4.34 [2.27, 6.58]	0.0002	0.0065
SAA (safe)	5,203	14.94	17.30	+2.36 [0.62, 4.06]	0.0094	0.0625
SAA (unsafe)	1,844	4.34	5.48	+1.14 [−0.81, 3.09]	0.25	0.49
TB, Qwen-generated	3,289	4.59	5.32	+0.73 [−1.73, 2.95]	0.53	0.55
TB, authors' samples	3,564	4.59	1.71	−2.88 [−5.21, −0.83]	0.0054	0.0002

Table 3: Replication with Qwen2.5-7B-Instruct-AWQ (clean run). Δ in error points. Baselines are computed on the scenarios each method covers (paired). TB is unpaired.

Safe items occasionally swap between safe answers that share a context, but the context is recovered in at least 99.98% of cases. We therefore cluster safe items by context and unsafe items by exact source.

6 Results

6.1 Replication on Qwen2.5-7B

Table 3 gives the overall results. **AKI** is by far the strongest method. Adversarial four-shot hints raise Qwen's error on unsafe scenarios from 4.6% to 24.6%, and the effect is significant in every domain. As in the paper's cross-model attacks, Household is the most affected domain (36.4% error; Appendix A). Zero-shot hints add 4.3 points. These AKI figures are conservative. Qwen often declined to write benefits and produced a warning instead, which was then injected as the hint. A keyword rule (phrases such as "none of", "no benefits", "is dangerous") flags 212 of the 1,181 hints (18%) as refusals or warnings. With the 969 remaining hints the four-shot error is 28.1%, against 9.0% with refusal hints (zero-shot: 10.2% vs. 3.3%). The ASSERT authors had annotators check benefit responses analogous to "there are no benefits"; our pipeline did not. **SAA** raises error on safe scenarios by 2.4 points. The paired bootstrap finds this significant ($p = 0.009$, which survives Holm's correction across the six overall tests), while the paper's z-test does not ($p = 0.06$). On unsafe scenarios SAA has no detectable effect. **TB** on Qwen's own, unfiltered scenarios has no detectable effect. On the authors' human-filtered scenarios Qwen's error *falls* by 2.9 points. The paper reports the same direction for GPT-3.5, Alpaca and Vicuna and attributes it to the filter keeping more overtly unsafe scenarios.

6.2 Re-analysis of the released outputs

The paper's numbers are recoverable. Comparing all method items against the full baseline, as the paper does, and scoring with the rule above, we reproduce all 16 overall effects in the paper's Tables 2–4 to within 0.03 points, the precision of its printed values (Table 4, columns 3–4). This confirms that our scoring and alignment match the authors' analysis.

		Paper-style (all items vs. full baseline)			Paired, scenario-level			
Model	Method	Paper Δ	Recomputed Δ	z-test p	Covered	Base %	Δ [95% CI]	p
GPT-3.5	SAA (safe)	+2.77	+2.76	0.0024	367 ctx	5.94	+2.76 [+1.48, +4.03]	<0.0001
	SAA (unsafe)	+1.78	+1.78	**0.2324**	370	5.95	+1.78 [+0.11, +3.46]	**0.0396**
	TB	−4.27	−4.26	<0.0001	–	5.95	−4.26 [−6.82, −1.89]	<0.0001
	AKI 0-shot	+7.60	+7.62	<0.0001	367	5.99	+7.57 [+5.01, +10.25]	<0.0001
	AKI 4-shot	+45.60	+45.61	<0.0001	367	5.99	+45.56 [+40.76, +50.14]	<0.0001
GPT-4	SAA (safe)	+0.45	+0.44	0.3658	367 ctx	1.83	+0.44 [−0.26, +1.11]	0.2194
	SAA (unsafe)	+0.81	+0.81	0.4120	370	2.43	+0.81 [−0.54, +2.16]	0.2614
	TB	+1.55	+1.55	0.1392	–	2.43	+1.55 [−0.22, +3.15]	0.0860
	AKI 0-shot	+17.30	+17.32	<0.0001	264	3.41	+16.35 [+12.48, +20.50]	<0.0001
	AKI 4-shot	+30.70	+30.71	<0.0001	264	3.41	+29.74 [+24.46, +35.10]	<0.0001
Alpaca	SAA (safe)	+7.26	+7.27	<0.0001	367 ctx	27.40	+7.27 [+5.22, +9.33]	<0.0001
	SAA (unsafe)	+1.52	+1.52	0.3013	370	5.95	+1.52 [−0.35, +3.41]	0.1218
	TB	−4.55	−4.54	<0.0001	–	5.95	−4.54 [−7.09, −2.11]	0.0002
Vicuna	SAA (safe)	+4.23	+4.24	0.0001	363 ctx	8.59	+4.32 [+2.86, +5.79]	<0.0001
	SAA (unsafe)	+7.27	+7.27	**0.0010**	**245**	**15.10**	+2.71 [−0.84, +6.22]	**0.1398**
	TB	−7.12	−7.12	<0.0001	–	10.54	−7.12 [−10.47, −3.94]	<0.0001

Table 4: Re-analysis of the ASSERT authors' released outputs. All Δ are in error points. The paper's Tables 2–3 report accuracy, so their sign is flipped here. "Covered" is the number of source scenarios ("ctx": contexts) with at least one generated item, out of 370 unsafe scenarios or 367 contexts. TB is unpaired and clustered by context. Bold marks the two conclusions that change.

Vicuna, SAA on unsafe scenarios. The paper reports a significant 7.27-point drop in accuracy, the only significant overall SAA effect on unsafe scenarios in its Table 2. Vicuna, however, produced usable paraphrases for only 245 of the 370 unsafe scenarios, and those are harder than average: Vicuna's original error on them is 15.1%, against 10.5% on all 370. Comparing the paraphrases with the full baseline mixes the method's effect with this change in which scenarios are tested. On the same 245 scenarios the effect is +2.7 points with a 95% interval that includes zero ($p = 0.14$). The result does not depend on the aligner: a cruder matcher that assigns each paraphrase to its highest-overlap source, with no ordering constraint, covers 250 scenarios and gives +3.0 points (95% CI $[-0.6, 6.7]$). Most of the published effect comes from which scenarios the test set covers, not from paraphrasing.

GPT-3.5, SAA on unsafe scenarios. Here coverage is complete, and the paired bootstrap is more powerful than the unpaired z-test. The same +1.78-point effect goes from $p = 0.23$ to $p = 0.04$. Under Holm's correction across the overall tests this result is not significant, so we read it as weak evidence of an effect rather than a reversal. It shows that the z-test is not uniformly too generous or too strict: on these data it erred in both directions.

Other results. AKI's large effects for GPT-3.5 and GPT-4 and SAA's effects on safe scenarios for GPT-3.5, Alpaca and Vicuna hold under every analysis. For GPT-4 AKI, the paired effect is about one point smaller than the published one (16.35 vs. 17.3 for zero-shot) because benefits exist for only 264 scenarios, which have a slightly higher baseline error.

Cross-model attacks (paper Table 5). From the released files we recompute every per-domain error rate for attacks on Alpaca to within 0.05 points (the overall GPT-3.5→Alpaca error is printed as 53.5% against 53.30% recomputed). The captions do not state which baseline Δ is measured against, but the reported change equals the target's error minus the *source* model's baseline, not Alpaca's. For example, Household with GPT-4 benefits: 72.8% error with a reported change of 72.0, implying a 0.8% baseline. That is GPT-4's Household baseline

(0.78%); Alpaca's is 6.98%. Overall, the change for GPT-4 benefits on Alpaca is +52.95 points against Alpaca's own baseline, not +56.5. For GPT-3.5 benefits the overall value is unchanged by chance, because the two models have the same overall baseline (5.95%), but per-domain values differ by up to 3.9 points. Vicuna's cross-model outputs were not released. One printed cell there appears inconsistent: the GPT-4→Vicuna overall error (66.2%) cannot be a weighted average of its four domain values (48.6–69.7%, weighted average 55.3%), whereas the reported change (52.8) equals 55.3 minus GPT-4's overall baseline. These corrections reduce, but do not change, the paper's conclusion that cross-model attacks are highly effective. We shared these observations with the first author before posting; at the time of writing we had not received a reply, and we will update this preprint if the authors confirm or correct them.

7 Discussion and recommendations

ASSERT's central finding, that injecting model-written justifications sharply increases unsafe answers, is robust. It holds for a 7B open model on free hardware and under every analysis of the original outputs. The weaker claims, about paraphrasing and bootstrapping, depend on analysis choices that are easy to get wrong. We suggest five practices for papers that evaluate on generated test sets.

1. **Compare on the same scenarios.** When generation fails for some scenarios, compute the baseline on the scenarios that remain, or at least report coverage and the baseline on the covered set.
2. **Resample scenarios, not items.** Variants of one scenario are correlated. A paired cluster bootstrap [3, 9] respects this and is usually more powerful than an unpaired test.
3. **Release source identifiers.** A source id on every generated item makes paired analysis trivial. Without one we had to reconstruct the link.
4. **Correct for multiple comparisons.** Per-domain tables contain dozens of tests, and isolated per-domain results at $p \approx 0.05$ are expected by chance.
5. **Report unparsed answers.** In the released files, up to 151 answers per condition (Vicuna, SAA safe) do not start with Yes or No. How they are counted changes error rates.

8 Limitations

We replicate on one 4-bit 7B model, so our numbers are not comparable in size with GPT-3.5 or GPT-4. vLLM's greedy decoding is not bit-identical across batch compositions, so generated sets differ slightly between runs; our two runs agree to within 0.7 points on every effect. Our TB scenarios were not filtered by humans, and some generated "unsafe" scenarios are plainly safe. The AKI hints were not checked by hand; the refusal rate above comes from a keyword rule and is approximate. Our alignment of Alpaca and Vicuna paraphrases cannot be checked directly. It is validated only by simulation on GPT-3.5 files, and safe items are aligned reliably only to the context level, which is why we cluster them by context. The bootstrap p-values are percentile approximations. Finally, the Vicuna cross-model observation relies on printed values only.

Ethics statement

SafeText and ASSERT contain descriptions of physically unsafe actions. We release code, prompts and aggregate statistics only, not the unsafe scenarios or benefits generated by Qwen. The authors' outputs remain in their repository. The aim of this work is to make safety evaluations more reliable.

Reproducibility

Code, the Colab notebook, the run manifest, integrity checks and all aggregate results are at https://github.com/rohailforty2/assert-qwen-repro. The re-analysis needs only the authors' public repository at commit `1cafa82` and runs on a CPU in under a minute. An interactive demo at https://rohailforty2.github.io/assert-qwen-repro/ replays, for 22 scenarios, the exact prompts sent to Qwen and its recorded answers, shows the released answers of the four original models for the same questions, and recomputes the scenario-level bootstrap in the browser from the per-scenario error counts.

Use of AI assistance

An AI assistant (Claude, Anthropic) was used to help write and review code, design the statistical analysis and draft this manuscript. The author ran all experiments, checked the results against the released files and the original paper, and takes full responsibility for the content.

Acknowledgements

We thank Alex Mei, Sharon Levy and William Yang Wang for releasing their code and model outputs and for permission to use their prompt text, and the SafeText and Qwen teams for releasing their data and models.

References

[1] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392, 2018.

[2] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, New York, 1993.

[3] Christopher A. Field and Alan H. Welsh. Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(3):369–390, 2007.

[4] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023. URL https://arxiv.org/abs/2309.06180.

[6] Sharon Levy, Emily Allaway, Melanie Subbiah, Lydia Chilton, Desmond Patton, Kathleen McKeown, and William Yang Wang. SafeText: A benchmark for exploring physical safety in language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 2407–2421, Abu Dhabi, United Arab Emirates, 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.154. URL https://aclanthology.org/2022.emnlp-main.154/.

[7] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware

weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023. URL https://arxiv.org/abs/2306.00978. Published at MLSys 2024.

[8] Alex Mei, Sharon Levy, and William Wang. ASSERT: Automated safety scenario red teaming for evaluating the robustness of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5831–5847, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.388. URL https://aclanthology.org/2023.findings-emnlp.388/.

[9] Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. *arXiv preprint arXiv:2411.00640*, 2024.

[10] Qwen, An Yang, Baosong Yang, Beichen Zhang, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. URL https://arxiv.org/abs/2412.15115.

A Per-domain AKI results on Qwen

Domain	Base err. %	4-shot err. %	Δ [95% CI]	Scenarios
Outdoors	4.88	19.05	+14.17 [+7.41, +21.49]	82
Medical	1.89	11.56	+9.68 [+4.07, +16.23]	106
Household	7.03	36.44	+29.41 [+22.63, +36.45]	128
Extra	3.92	26.42	+22.49 [+11.39, +34.64]	51
Overall	4.63	24.64	+20.01 [+16.11, +24.04]	367

Table 5: AKI adversarial four-shot on Qwen2.5-7B-Instruct-AWQ, by domain. All bootstrap $p < 0.0001$.